

630-Hembree_Alex_Exercise10.2

July 5, 2026

0.0.1 DSC 630 - Predictive Analytics

0.0.2 Week 10, Exercise 10.2

0.0.3 Hembree, Lex

Using the small MovieLens data set, create a recommender system that allows users to input a movie they like (in the data set) and recommends ten other movies for them to watch. In your write-up, clearly explain the recommender system process and all steps performed. If you are using a method found online, be sure to reference the source.

IMPORTANT: Please make sure to follow the following submission instructions carefully or it will result in having to resubmit or losing points if repeated incorrect submissions are made.

Submission Instructions:

- The assignment is due by Sunday, 11:59 p.m. CT.
- You need to submit the code file (your choice of R - .rmd file or Python - .ipynb file) and its PDF version (you can knit your .rmd file into a PDF file if you use R, or print or save .ipynb file as a PDF file if you use Python) and submit both files to Blackboard in the same submission.
- If you use Jupyter notebook, you need to save or print it as a PDF and submit along with the original notebook (or markdown file). Blackboard cannot render Jupyter notebook, that's why PDF (or Word doc) is required.
- Make sure to comment all your code as you are documenting your steps, process, and analysis.
- Please do not submit a hyperlink to your GitHub repositories or your Google account, just the above files. You are welcome to use GitHub and it is recommended to build a portfolio, but suggested you do so with a private repository and is not necessary to share your linked files.
- Please include the data with your submission if it's not given in the assignment.
- Please save your files with your first and last name and assignment number with each file you submit to Blackboard.

Gotta read it in else we won't get anywhere on the movie recommendations (unless you ask me and then I can probably give some good ones).

```
[1]: # import libraries
import numpy as np
import pandas as pd

# import file as pandas data frame
```

```
links_df = pd.read_csv("C:/Users/Ashem/OneDrive/Desktop/Masters Program - DSC/
↳DSC 630 - Predictive Analytics/Week 10/links.csv")
movies_df = pd.read_csv("C:/Users/Ashem/OneDrive/Desktop/Masters Program - DSC/
↳DSC 630 - Predictive Analytics/Week 10/movies.csv")
ratings_df = pd.read_csv("C:/Users/Ashem/OneDrive/Desktop/Masters Program - DSC/
↳DSC 630 - Predictive Analytics/Week 10/ratings.csv")
tags_df = pd.read_csv("C:/Users/Ashem/OneDrive/Desktop/Masters Program - DSC/
↳DSC 630 - Predictive Analytics/Week 10/tags.csv")
```

Preview the dataframes to see what we can work with (this would have been an amazing thing when I was younger)

```
[2]: # Preview
links_df.head()
```

```
[2]:   movieId  imdbId   tmdbId
0        1  114709    862.0
1        2  113497    8844.0
2        3  113228   15602.0
3        4  114885   31357.0
4        5  113041   11862.0
```

```
[3]: # Preview
movies_df.head()
```

```
[3]:   movieId                                     title \
0        1                               Toy Story (1995)
1        2                               Jumanji (1995)
2        3          Grumpier Old Men (1995)
3        4          Waiting to Exhale (1995)
4        5  Father of the Bride Part II (1995)

      genres
0  Adventure|Animation|Children|Comedy|Fantasy
1          Adventure|Children|Fantasy
2              Comedy|Romance
3          Comedy|Drama|Romance
4              Comedy
```

```
[4]: # Preview
ratings_df.head()
```

```
[4]:   userId  movieId  rating  timestamp
0        1         1     4.0   964982703
1        1         3     4.0   964981247
2        1         6     4.0   964982224
3        1        47     5.0   964983815
4        1        50     5.0   964982931
```

```
[5]: # Preview
tags_df.head()
```

```
[5]:   userId  movieId      tag  timestamp
0      2    60756    funny  1445714994
1      2    60756  Highly quotable  1445714996
2      2    60756  will ferrell  1445714992
3      2    89774  Boxing story  1445715207
4      2    89774      MMA  1445715200
```

Merge the dataframes into one (good ole days of burning cd's or movies)

```
[6]: # merge tags with movie titles
movies_tags = tags_df.merge(movies_df, on='movieId')
```

Collapse the Tags (forget everything you think you know)

```
[7]: # collapse all tags for each movie into one string
movie_tag_text = movies_tags.groupby(['movieId', 'title'])['tag'].apply(
    lambda x: " ".join(x.astype(str))
).reset_index()
```

Build the matrix (red or blue?)

```
[8]: # import library
from sklearn.feature_extraction.text import TfidfVectorizer

# build TF-IDF matrix for tag text
tfidf = TfidfVectorizer(stop_words='english')
tag_matrix = tfidf.fit_transform(movie_tag_text['tag'])
```

Correlate the similarities (deja vu is the change in the program)

```
[9]: # import library
from sklearn.metrics.pairwise import cosine_similarity

# compute tag-based similarity
tag_similarity = cosine_similarity(tag_matrix)
tag_similarity_df = pd.DataFrame(
    tag_similarity,
    index=movie_tag_text['title'],
    columns=movie_tag_text['title']
)
```

Build rating matrix (isn't that what Agent Smith really wanted?)

```
[10]: # build user-movie rating matrix
ratings_merged = ratings_df.merge(movies_df, on='movieId')
user_movie_matrix = ratings_merged.pivot_table(
    index='userId',
    columns='title',
```

```
        values='rating'  
    ).fillna(0)
```

Correlate the rating similarities (fan theory is that Agent Smith was “The One”)

```
[11]: # compute rating-based similarity  
rating_similarity = cosine_similarity(user_movie_matrix.T)  
rating_similarity_df = pd.DataFrame(  
    rating_similarity,  
    index=user_movie_matrix.columns,  
    columns=user_movie_matrix.columns  
)
```

Align the two matrixes (Rewatch and compare Nea and Smith, it’s interesting)

```
[12]: # align the two similarity matrices so they share the same movie index  
common_movies = tag_similarity_df.index.intersection(rating_similarity_df.index)  
tag_sim = tag_similarity_df.loc[common_movies, common_movies]  
rating_sim = rating_similarity_df.loc[common_movies, common_movies]
```

Normalize (The reloaded scene where they said “Neo is doing his superman thing”)

```
[13]: # normalize both similarity matrices to the same scale  
tag_norm = (tag_sim - tag_sim.min().min()) / (tag_sim.max().max() - tag_sim.  
    ↪min().min())  
rating_norm = (rating_sim - rating_sim.min().min()) / (rating_sim.max().max() -  
    ↪rating_sim.min().min())  
  
# blend the two similarity signals (50/50 weight, can be adjusted)  
hybrid_similarity = 0.5 * tag_norm + 0.5 * rating_norm
```

Engage recommender (Plugging into the Machines to stop Smith one last time) (Note: This recommender system uses standard TF-IDF and cosine similarity methods commonly used in content-based and collaborative filtering systems.)

```
[14]: # recommender function using the hybrid similarity  
def recommend_hybrid(movie_title, num_recs=10):  
    if movie_title not in hybrid_similarity.columns:  
        print(f"'{movie_title}' not found in the dataset.")  
        return  
  
    similarity_scores = hybrid_similarity[movie_title]  
    similar_movies = similarity_scores.sort_values(ascending=False)  
    recommendations = similar_movies.iloc[1:num_recs+1]  
  
    df = recommendations.reset_index()  
    df.columns = ['title', 'similarity']  
    df['similarity'] = df['similarity'].round(3)
```

```
print(df.to_string(index=False))
```

Showcase (the final fight)

```
[15]: # example  
recommend_hybrid("Toy Story (1995)")
```

	title	similarity
	Bug's Life, A (1998)	0.691
	Toy Story 2 (1999)	0.580
	Guardians of the Galaxy 2 (2017)	0.285
	Jurassic Park (1993)	0.283
	Independence Day (a.k.a. ID4) (1996)	0.282
	Up (2009)	0.282
	Star Wars: Episode IV - A New Hope (1977)	0.279
	Forrest Gump (1994)	0.274
	Lion King, The (1994)	0.271
Star Wars: Episode VI - Return of the Jedi (1983)		0.271